Module 07 — Documentation

Tier: 🟢 Beginner · **Duration:** 60 min · **Prerequisites:** Module 06

> ***Framing:*** *Documentation isn't an afterthought. CI fails if Silver or Gold models have missing descriptions or undocumented columns. This module closes the Beginner tier by treating documentation as a non-negotiable part of the development workflow — not something you add before merging, but something you write while you're building.*

Agenda

Time	Duration	Topic	Learning Goal	Mode	Participant Activity	Materials	
00:00	10 min	Recap Module 06	Confirm testing mental model	Q&A	Answer from memory	—	Ask all 4 "what do different `test`?"
00:10	10 min	Why documentation is a CI requirement	Understand the business and operational cost of missing docs	Present	Listen	This doc	Use a rea and asks `fct_pres` Is it a UU the colu nobody reading
00:20	15 min	Writing model and column descriptions	Write correct `schema.yml` descriptions including the grain statement	Present + live code	Follow along in editor	This doc	Write a f including descripti descripti same file separate
00:35	10 min	`persist_docs` — docs in Snowflake	Know that dbt writes descriptions to Snowflake column comments	Present	Annotate config	This doc	Show Sn over a co the dbt c is why Po read colu
00:45	5 min	`dbt docs generate` and `dbt docs serve`	Know how to build and browse the doc site	Demo	Watch	VS Code terminal	Run both the DAG lineage v descripti

00:50	20 min	Exercise: document a real model	Write complete documentation for an undocumented model	Practice	Solo exercise	Exercise below	Circulate writing c not for t sure the present.
01:10	10 min	Debrief + Beginner tier close	Consolidate and acknowledge completion of Tier 1	Debrief	Verbal	—	Explicitly now com You can documer what Int

---

Content

Part A — Why Documentation Is Mandatory

Three months after a model is built, the original author can't remember what `fct_prescription.dosage_amount` represents. Is it in milligrams? Is it a monetary amount? Is NULL valid?

If that column isn't documented in `schema.yml`, anyone who needs to know must read the SQL, trace it back through staging to Bronze, and hope the source system has docs. That's hours of investigation for information that could have been one sentence.

This isn't acceptable for Silver or Gold models. CI fails if:

- A Silver or Gold model has no description
- A Silver or Gold model has no grain statement
- Any column in a Silver or Gold model has no description

You write documentation once. It saves everyone who touches the model afterward from having to figure it out themselves.

Part B — Writing Model and Column Descriptions

```yaml
version: 2

models:
  - name: fct_prescription
    description: >
      Grain: one prescription event per patient per doctor per prescription_date.
      Source: HubSpot deal records filtered to prescription pipeline stages.
      Refreshed nightly. Excludes cancelled prescriptions.
    columns:
      - name: prescription_key
        description: >
          Surrogate primary key. MD5 hash of (patient_key, doctor_key, prescription_date).
          Integer. Not a UUID.
        tests:
```

```
          - unique
          - not_null

      - name: patient_key
        description: "FK to dim_patient.patient_key. Surrogate key generated by our
pipeline."
        tests:
          - not_null
          - relationships:
              to: ref('dim_patient')
              field: patient_key

      - name: doctor_key
        description: "FK to dim_doctor.doctor_key."
        tests:
          - not_null
          - relationships:
              to: ref('dim_doctor')
              field: doctor_key

      - name: prescription_date
        description: "Date the prescription was issued. No time component."
        tests:
          - not_null

      - name: medication_type
        description: "Category of medication dispensed. Values: tablet, liquid, injection,
topical."
        tests:
          - accepted_values:
              values: ['tablet', 'liquid', 'injection', 'topical']

      - name: dosage_amount
        description: "Prescribed dosage in milligrams. Null if not yet confirmed by the
clinic."
        tests:
          - not_null:
              severity: warn

      - name: notes
        description: "Optional free-text notes added by the prescribing doctor. May be
null."
```

The grain statement is mandatory for Silver and Gold. It answers: one row = what?

Grain statement format:

```
Grain: one {entity} per {dimension1} per {dimension2} per {time dimension}.
```

If you can't write the grain statement, the model's design is unclear. Review the design before you document it.

Doc blocks — for longer descriptions

When a description runs longer than a sentence or two, move it into a **doc block** in a separate `.md` file. This keeps the YAML clean.

```
<!-- models/silver/_silver_docs.md -->
{% docs fct_prescription %}
One prescription event per patient per doctor per prescription_date.
Source: HubSpot deal records filtered to the prescription pipeline.
Excludes cancelled and draft prescriptions (status != 'closed_won').
Refreshed nightly after the Bronze load completes.
{% enddocs %}
```

Then reference it in `schema.yml`:

```
models:
  - name: fct_prescription
    description: '{{ doc("fct_prescription") }}'
```

Short inline descriptions are standard for most columns. Use doc blocks only for models where the business context genuinely needs more than two lines — typically complex Silver facts or models with non-obvious exclusion logic.

Part C — `persist_docs` — Writing Descriptions to Snowflake

This is already configured in `dbt_project.yml` for Silver and Gold:

```
models:
  analytics:
    silver:
      +persist_docs:
        relation: true    # writes model description to Snowflake table comment
        columns: true     # writes column descriptions to Snowflake column comments
```

What this means: When `dbt run` or `dbt build` executes a Silver model, dbt also runs:

```
COMMENT ON TABLE SILVER.PUBLIC.fct_prescription IS 'Grain: one prescription event per...';
COMMENT ON COLUMN SILVER.PUBLIC.fct_prescription.prescription_key IS 'Surrogate primary
key...';
```

These comments are visible in:

- **Snowsight** — hover over a column in the table browser
- **Power BI** — column tooltips and field descriptions
- **Snowflake Cortex AI** — used as context for natural language queries

That's why documentation isn't optional — it feeds directly into tools the business uses every day.

Part D — Building and Browsing the Docs Site

```
# Step 1: Generate the docs artifact
dbt docs generate
```

```
# Step 2: Serve locally
dbt docs serve
# Opens browser at http://localhost:8080
```

What you'll find at `localhost:8080`:

- **Model list** — all models with their descriptions
- **DAG view** — full lineage from Bronze sources through to Gold marts
- **Column panel** — column descriptions, types, tests
- **Source freshness** — last run status per source

The DAG is the fastest way to answer: "if I change `dim_patient`, what downstream models are affected?"

Exercise (20 min)

> ***Project context:*** *`models/silver/schema.yml` exists from Module 06 with tests only. This session extends it with descriptions and grain statements, completing the CI requirements for Silver.*

Task 1 — Add documentation to `fct_prescription`

In the existing `models/silver/schema.yml`, add a `description:` field to the `fct_prescription` model. The description must include:

- A grain statement: what does one row represent?
- One sentence on what the model is used for

Then add `description:` to each column that already has tests.

Task 2 — Document `dim_pipeline`

Add a new model entry for `dim_pipeline` to `schema.yml`. This is an SCD2 model — look at `data/silver/dim_pipeline.csv` to see a concrete example: `hs-pipeline-003` appears twice, showing the before and after of a pipeline rename.

Column	Type	Role
`pipeline_key`	Surrogate PK	Unique per pipeline version
`hubspot_pipeline_id`	String	Business key — stable across SCD2 versions
`pipeline_name`	String	Human-readable name
`is_active`	Boolean	Whether pipeline is in use
`dbt_valid_from`	Timestamp	When this version became effective
`dbt_valid_to`	Timestamp	When superseded. NULL = current
`is_current`	Boolean	True if `dbt_valid_to IS NULL`

Requirements:

- Grain statement that explains the SCD2 pattern
- Description for every column

- Tests: `pipeline_key` (unique + not_null, error), `is_current` (not_null, error)

Task 3 — Generate and browse the docs

```
dbt docs generate
dbt docs serve
```

In the browser:

1. Navigate to `fct_prescription`. Confirm the grain statement appears in the model description.
2. Navigate to `dim_pipeline`. Click into the DAG. Find the lineage upstream to Bronze.
3. Click the `dbt_valid_to` column. Confirm your description renders.
4. Trace the full lineage from `BRONZE.HUBSPOT.contacts` through to `fct_prescription`.

Reference Material

- dbt documentation — schema.yml
- dbt `persist_docs` config
- Internal: `conventions` skill — documentation requirements per layer, grain format
- Internal: `references/go_live_checklist.md` — pre-merge checklist including documentation requirements

---

Beginner Tier Complete

You've now completed all 7 modules in the Beginner tier. You can:

- Explain what dbt is and what it does
- Navigate the project structure and understand `dbt_project.yml` and `profiles.yml`
- Read and write Jinja in dbt models: `{{ ref() }}`, `{{ source() }}`, `{{ config() }}`, `{% if %}`
- Choose the correct materialization for a given use case
- Declare sources and write staging models that reference them correctly
- Write a complete test suite for a Silver model using all four generic tests
- Write model and column documentation that passes CI requirements
- Use `dbt build`, `dbt docs serve`, and `dbt source freshness`

Intermediate tier (Modules 08–13) covers: advanced incremental patterns, SCD2 and the `scd2_merge` macro, Jinja macros and packages, seeds and variables, selectors and tags, and CI/CD with slim CI.

Prep Questions for Module 08 (Intermediate — Advanced Incremental)

1. What is `on_schema_change: sync_all_columns` and why is it required?
2. When would you use `--full-refresh` on an incremental model?
3. What SQL statement does an incremental model with `unique_key` generate?
4. What's the difference between `_key` and `_id` columns?